Final Prject

65090502410 นายณภัทร เกียรติกอบลาภ, 65090502414 นายธนวินท์ แสงวิสุทธิ์ ,65090502417 ปิยพัทธ์ จันทร์ทวี

2025-11-28

ที่มาและความสำคัญ

อุบัติเหตุทางถนนในรัฐเท็กซัส ประเทศสหรัฐอเมริกายังคงเป็นปัญหารุนแรงเนื่องจากปริมาณการจราจรที่หนาแน่นและพื้นที่ขนาดใหญ่ การวิเคราะห์ปัจจัยที่ส่งผลต่อการเกิดและความรุนแรงของอุบัติเหตุจึงมีความสำคัญต่อการวางแผนลดความเสี่ยง การใช้ Machine Learning สามารถช่วยพยากรณ์ความรุนแรงของอุบัติเหตุ ระบุพื้นที่และช่วงเวลาที่เสี่ยงสูง และสนับสนุนการจัดสรรทรัพยากรและการตอบสนองเหตุการณ์ได้อย่างมีประสิทธิภาพ

วัตถุประสงค์

1.) เพื่อสร้างโมเดลการทำนายความรุนแรงของอุบัติเหตุทางถนนในรัฐเท็กซัส ประเทศสหรัฐอเมริกา 2.) เพื่อคาดการณ์และระบุจุดเสี่ยงที่มีความรุนแรงสูงต่อการเกิดอุบัติเหตุ 3.) เพื่อประเมินและวิเคราะห์ปัจจัยที่มีผลต่อความรุนแรงของอุบัติเหตุ

ประโยชน์ที่คาดว่าจะได้รับ

1.) ช่วยในการวางแผนปรับปรุงสัญญาณจราจรและมาตรการความปลอดภัยในพื้นที่ที่มีความเสี่ยงสูง 2.) ช่วยลดการจราจรติดขัดและความเสียหายทางเศรษฐกิจที่เกิดจากอุบัติเหตุร้ายแรง 3.) เพื่อประเมินและวิเคราะห์ปัจจัยที่มีผลต่อความรุนแรงของอุบัติเหตุ

ชุดข้อมูลที่ใช้

ชุดข้อมูลอุบัติเหตุรถยนต์ทั่วประเทศ ครอบคลุม 49 รัฐของสหรัฐอเมริกาข้อมูลอุบัติเหตุถูกเก็บรวบรวมตั้งแต่เดือนกุมภาพันธ์ 2559 ถึงมีนาคม 2566 ชุดข้อมูลนี้มีบันทึกอุบัติเหตุประมาณ7.7 ล้านรายการ จาก https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents (https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents) เนื่องจากข้อมูลมีปริมาณมากจึงจะเจาะจงไปที่รัฐ Texas เมือง San Antonio

วิธีการดำเนินการ

1.) เตรียม Package และข้อมูลที่จะใช้
2.) จัดการเตรียมข้อมูลและกำหนดตัวแปร
3.) ทำ Unsupervise ด้วย PCA และ ใช้ k-means clustering และ Expectation-Maximization clustering (EM clustering) โดยเลือกจำนวนจุด cluster ด้วย library(UniversalCVI) พร้อมตีความคุณลักษณะของแต่ละ cluster ผลจากการทำ cluster จะนำไปใช้เป็นหนึ่งในตัวแปรของการทำ Supervise
4.) ทำ Supervise แบ่ง training set และ test set สร้างตัวแบบ Random Forest, XGboost, BART และทำ Cross-Validation ด้วย 5-fold และเปรียบเทียบด้วย ค่า F1-score
5.) วิเคราะห์ตัวแปรจากตัวแบบที่ดีที่สุด
6.) เลือกจุดเกิดเหตุยอดนิยมที่มีผลกับความรุนแรงอุบัติเหตุมา 3 จุดและ หา Wait time ของการเกิดอุบัติเหตุต่อไป ณ บริเวณนั้นๆ

.

ตัวแปรที่ใช้

Start_Time เวลาที่อุบัติเหตุเริ่มต้น (ตามเวลาท้องถิ่น) Severity ระดับความรุนแรงของอุบัติเหตุ (1 = น้อยสุด, 4 = รุนแรงสุด) Start_Lat ละติจูด (GPS) จุดเริ่มต้นของอุบัติเหตุ Start_Lng ลองจิจูด (GPS) จุดเริ่มต้นของอุบัติเหตุ Temperature(F) อุณหภูมิ (ฟาเรนไฮต์) Wind_Chill(F) อุณหภูมิที่รู้สึกได้จากลม Humidity(%) ความชื้นสัมพัทธ์ (%) Pressure(in) ความดันอากาศ (นิ้วปรอท) Visibility(mi) ทัศนวิสัย (หน่วยไมล์) Wind_Speed(mph) ความเร็วลม (ไมล์ต่อชั่วโมง) Precipitation(in) ปริมาณฝนหรือหิมะตก (นิ้ว) Amenity มีสิ่งอำนวยความสะดวกอยู่ใกล้หรือไม่ Bump มีลูกระนาดหรือไม่ Crossing มีทางข้ามถนนอยู่ใกล้หรือไม่ Give_Way มีป้าย “ให้ทาง (Give Way)” หรือไม่ Junction อยู่ใกล้สี่แยกหรือไม่ No_Exit อยู่ในถนนตันหรือไม่ Railway อยู่ใกล้ทางรถไฟหรือไม่ Roundabout อยู่ใกล้วงเวียนหรือไม่ Station อยู่ใกล้สถานี (รถไฟ/รถบัส) หรือไม่ Stop อยู่ใกล้ป้ายหยุดหรือไม่ Traffic_Calming อยู่ในพื้นที่ที่มีการชะลอความเร็วหรือไม่ Traffic_Signal มีสัญญาณไฟจราจรอยู่ใกล้หรือไม่ Sunrise_Sunset ช่วงเวลา (กลางวัน/กลางคืน) ตามดวงอาทิตย์ขึ้น/ตก Civil_Twilight ช่วงเวลาเช้า/ค่ำ ตามนิยามพลเรือน
Nautical_Twilight ช่วงเวลาเช้า/ค่ำ ตามนิยามเดินเรือ
Astronomical_Twilight ช่วงเวลาเช้า/ค่ำ ตามนิยามดาราศาสตร์

1.Load Libraries and Data

จากการจำนวนข้อมูลที่มีขนาดใหญ่และสัดส่วนของ class imbalance จึงจะมีการสุ่มข้อมูลบางส่วนและทำ over sampling ร่วม โดยการ สุ่ม 20% จาก row ที่ class = 1 และ 2 ,สุ่ม 50% จาก row ที่ class = 3 และ 4

```
table(texas$Severity)
```

```
##
##     1     2     3     4
##   301 28018  8630   369
```

```
set.seed(123)
dat <- bind_rows(
  texas |>
    filter(Severity %in% c(1,2)) |>
    slice_sample(prop = 0.2),

  texas |>
    filter(Severity %in% c(3,4)) |>
    slice_sample(prop = 0.5)
)
```

โดยจะได้สัดส่วน class ดังนี้

```
table(dat$Severity)
```

```
##
##    1    2    3    4
##   60 5603 4313  186
```

```
round(prop.table(table(dat$Severity)) * 100, 2)
```

```
##
##     1     2     3     4
##  0.59 55.14 42.44  1.83
```

2. Data Preprocessing for Classification

เลือกตัวแปรที่จะนำมาใช้งาน cleaning data และจากการที่ Severity 1 และ 4 (Class 1 และ 4) มีสัดส่วนที่น้อยแม้จะเป็นชุดข้อมูลก่อนมีการสุ่มบางส่วน (San Antonio Severity 1 รวม 4 ได้อยู่ที่ 670 case จาก 37,318) จึงจะรวม ระดับ Severity เหลือเพียง 2 class (0 = ไม่รุนแรง, 1 = รุนแรง) โดย class_0 = Severity 1+2, class_1 = Severity = 3+4 และ แปลงตัวแปร charactor ให้พร้อมสำหรับการนำไปใช้งาน

```
# Select relevant columns and remove NA
texas_rf <- dat %>%
mutate(Severity = as.factor(Severity)) %>%
select(Severity, Start_Lat, Start_Lng, 21:46) %>%
na.omit()

# Convert logical/character/factor to numeric for ML

texas_clean <- texas_rf %>%
mutate(across(where(is.logical), as.numeric),
across(where(is.character), ~ as.numeric(factor(.))),
across(where(is.factor) & !matches("Severity"), ~ as.numeric(.))) %>%
select(-Turning_Loop, -Weather_Condition, -Wind_Direction) %>%
select(where(~ n_distinct(.) > 1) | Severity) %>%
na.omit()

# Convert twilight features

texas_clean <- texas_clean %>%
mutate(Sunrise_Sunset = Sunrise_Sunset - 1,
Civil_Twilight = Civil_Twilight - 1,
Nautical_Twilight = Nautical_Twilight - 1,
Astronomical_Twilight = Astronomical_Twilight - 1)

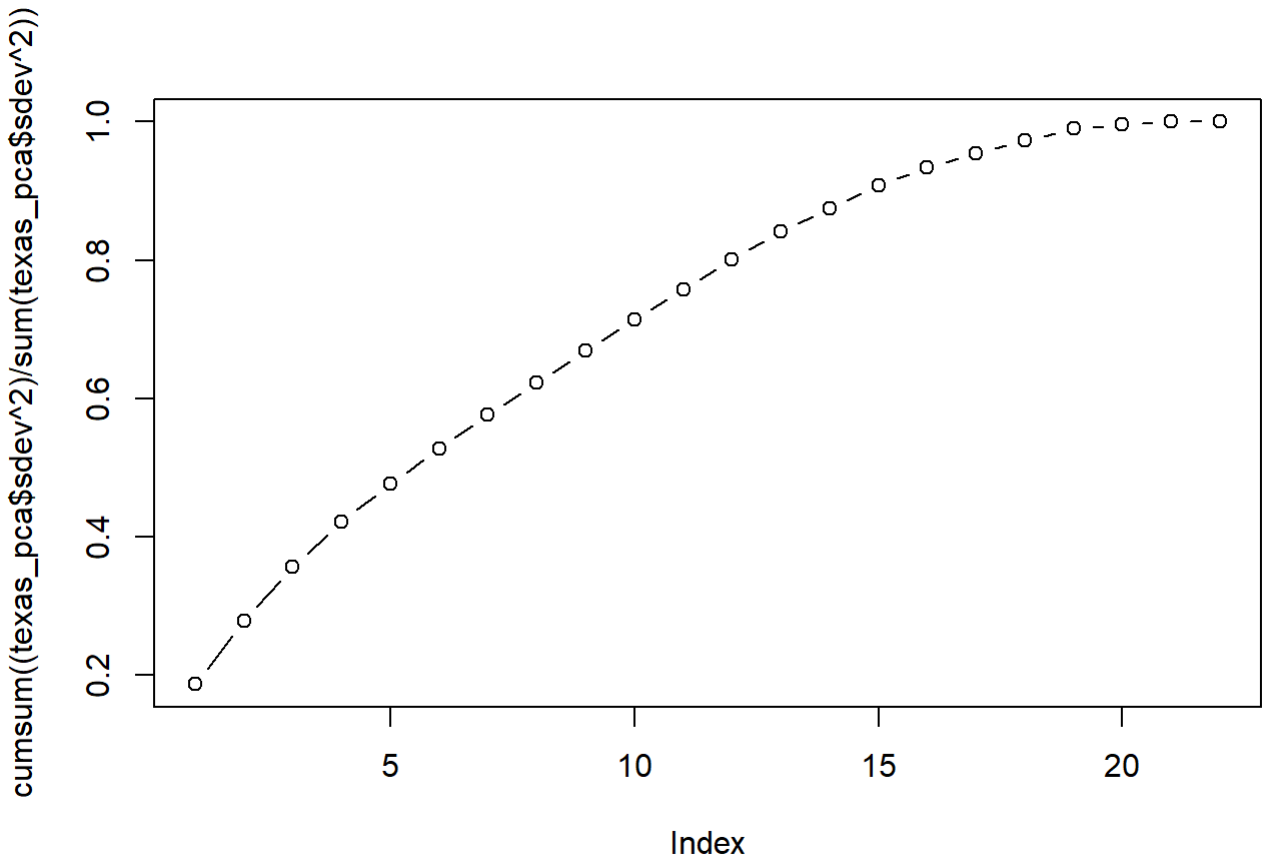
# Binary severity: 0 = low (1,2), 1 = high (3,4)

texas_clean$Severity <- ifelse(texas_clean$Severity %in% c(1,2), 0, 1)
```

3. Unsupervise

ทำ PCA และ Kmeans

```
texas_pca <- prcomp(texas_clean |> select(-Severity) |> scale())
plot(cumsum((texas_pca$sdev^2)/sum(texas_pca$sdev^2)), type = "b")
```



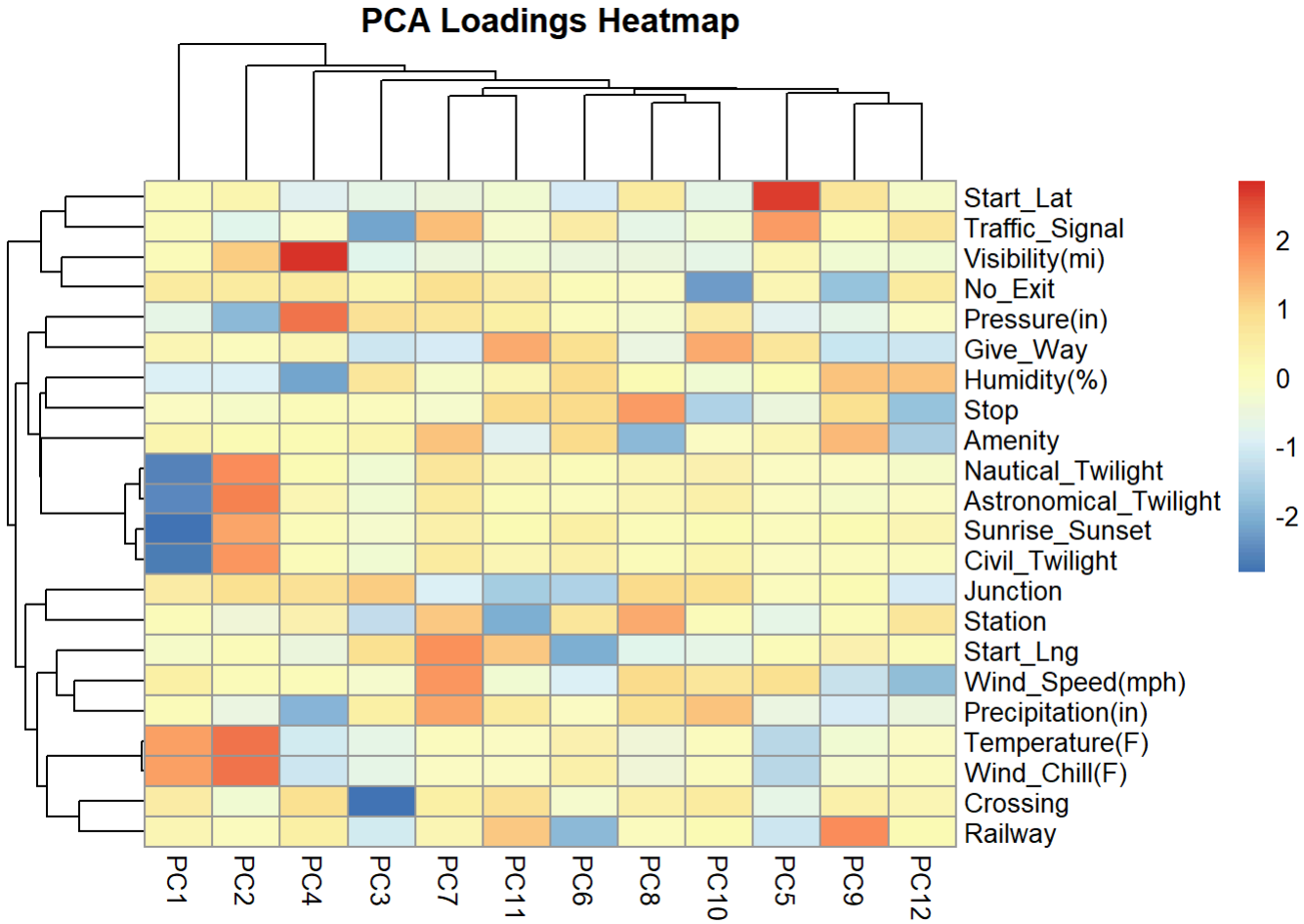

```
pc_select <- which(cumsum(texas_pca$sdev^2/sum(texas_pca$sdev^2)) > 0.8)[1]

texas_pca$rotation[, 1:pc_select]
```

```
##                                PC1          PC2          PC3          PC4
## Start_Lat              0.009774295  0.073632728 -0.168895292 -0.192127557
## Start_Lng              0.016897942  0.068841737  0.228198196 -0.043487205
## Temperature(F)         0.337759865  0.425241082 -0.077789170 -0.150790497
## Wind_Chill(F)          0.333112045  0.429714833 -0.079918658 -0.160632732
## Humidity(%)           -0.231930740 -0.227830625  0.016507027 -0.421322301
## Pressure(in)          -0.119686006 -0.371500252  0.171999057  0.446709775
## Visibility(mi)         0.091610632  0.274448271 -0.041526196  0.559406367
## Wind_Speed(mph)        0.139872872  0.069723351 -0.016247646  0.046618727
## Precipitation(in)     -0.024688626 -0.165180358  0.060651970 -0.453427748
## Amenity                0.008311690 -0.030568583  0.009058684 -0.019776095
## Crossing               0.004816467 -0.141225946 -0.602897830  0.061082113
## Give_Way              -0.002140053 -0.061826305 -0.329248051 -0.004856625
## Junction              -0.019754782  0.065851061  0.119370036  0.039048658
## No_Exit                0.006754932  0.007605909 -0.085494797  0.005624598
## Railway               -0.002863728 -0.059854333 -0.320664230  0.041389365
## Station                0.006475983 -0.107256284 -0.313917348  0.075152090
## Stop                  -0.005744008 -0.040802114  0.005471157  0.036691579
## Traffic_Signal         0.029246017 -0.129176136 -0.401801059 -0.008110697
## Sunrise_Sunset        -0.414440327  0.202761959 -0.056530080 -0.007007785
## Civil_Twilight        -0.425844406  0.256252303 -0.064285477 -0.003651018
## Nautical_Twilight     -0.416813041  0.282379695 -0.069305468  0.008319794
## Astronomical_Twilight -0.386758243  0.279137322 -0.054154347  0.016758046
##                                PC5          PC6          PC7          PC8
## Start_Lat              0.65869657 -0.242102587 -0.127711579  0.147224234
## Start_Lng              0.05897549 -0.390435788  0.434527639 -0.103917956
## Temperature(F)        -0.21347174  0.097453723  0.037840933 -0.034319472
## Wind_Chill(F)         -0.22031805  0.104130774  0.024149286 -0.041349712
## Humidity(%)           -0.06690604  0.051182183 -0.120718162 -0.070499124
## Pressure(in)          -0.15264637  0.010929787  0.148539791 -0.038820322
## Visibility(mi)         0.12399688 -0.010465806  0.002023927 -0.002518075
## Wind_Speed(mph)        0.22933948 -0.176255651  0.439700035  0.257191104
## Precipitation(in)     -0.16389730 -0.068206234  0.301428791  0.156717992
## Amenity               -0.01378426  0.209463271  0.280980054 -0.610077588
## Crossing              -0.21082005 -0.133980698 -0.005425873 -0.015480978
## Give_Way               0.12486963  0.139474060 -0.307740445 -0.200034364
## Junction              -0.14167710 -0.476033145 -0.346068086  0.085059917
## No_Exit               -0.09186290 -0.128071372  0.072530457 -0.174578540
## Railway               -0.32557167 -0.523532631 -0.003221431 -0.064293942
## Station               -0.16867412  0.161875779  0.291026319  0.375771634
## Stop                  -0.11754690  0.300708849 -0.055131809  0.507143726
## Traffic_Signal         0.32515511  0.105263796  0.255430730 -0.117827152
## Sunrise_Sunset        -0.02128540  0.046031124  0.032888891 -0.004816814
## Civil_Twilight        -0.03112188  0.040514175  0.068204410  0.003781129
## Nautical_Twilight     -0.03094074  0.007267037  0.088894752  0.023622153
## Astronomical_Twilight -0.02846645 -0.011872837  0.081791283  0.016574692
##                                PC9         PC10         PC11          PC12
## Start_Lat              0.172292409 -0.16179773 -0.071209060 -0.0505517618
## Start_Lng              0.117217389 -0.10093948  0.305631756  0.0590155669
## Temperature(F)        -0.012866562  0.03565723  0.028278268  0.0295309790
## Wind_Chill(F)         -0.006251929  0.03012505  0.027353391  0.0384014116
## Humidity(%)            0.098066280 -0.13628820 -0.059048446  0.0940301040
## Pressure(in)          -0.112902881  0.11255985  0.100629087 -0.0152931381
## Visibility(mi)         0.023805985 -0.03634188  0.027509629  0.0294605009
## Wind_Speed(mph)       -0.236481930  0.20050370 -0.029963467 -0.3829751038
```

```
## Precipitation(in)     -0.245335696  0.23639447  0.100455960 -0.1421277154
## Amenity                0.313213399 -0.08575963 -0.315846636 -0.5130633349
## Crossing              -0.020242389  0.01662643  0.047890682 -0.0461352397
## Give_Way              -0.346408745  0.31257365  0.320472340 -0.3358991731
## Junction              -0.102641889  0.06336719 -0.512347642 -0.3568446982
## No_Exit               -0.602228504 -0.73226225 -0.027360380  0.0001545881
## Railway                0.390201181 -0.02697626  0.234874982 -0.0185480159
## Station                0.027304846  0.01174358 -0.511803078  0.1751653313
## Stop                   0.261736615 -0.42331868  0.295755762 -0.5054928243
## Traffic_Signal         0.027561863 -0.04876853 -0.032826217  0.1403216838
## Sunrise_Sunset         0.001693579  0.00570393  0.001784702  0.0088418511
## Civil_Twilight        -0.015542475  0.02486810  0.022464882 -0.0159615600
## Nautical_Twilight     -0.035681951  0.04330759  0.020350874 -0.0406376815
## Astronomical_Twilight -0.042601490  0.04773578  0.005308938 -0.0323717375
```

```
pheatmap(
  texas_pca$rotation[, 1:pc_select],
  scale = "row",
  main = "PCA Loadings Heatmap"
)
```

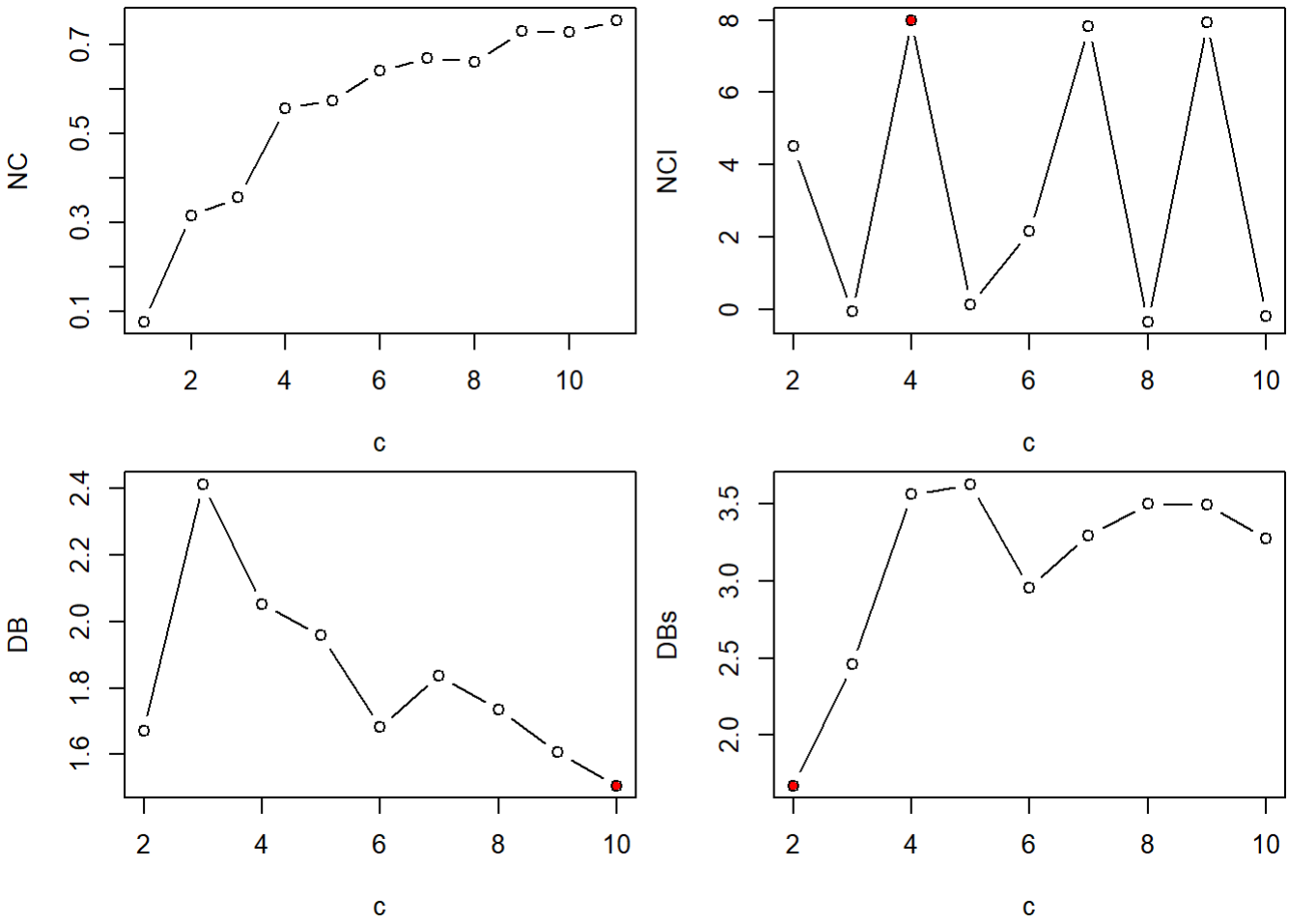


เลือกจำนวนกลุ่ม cluster

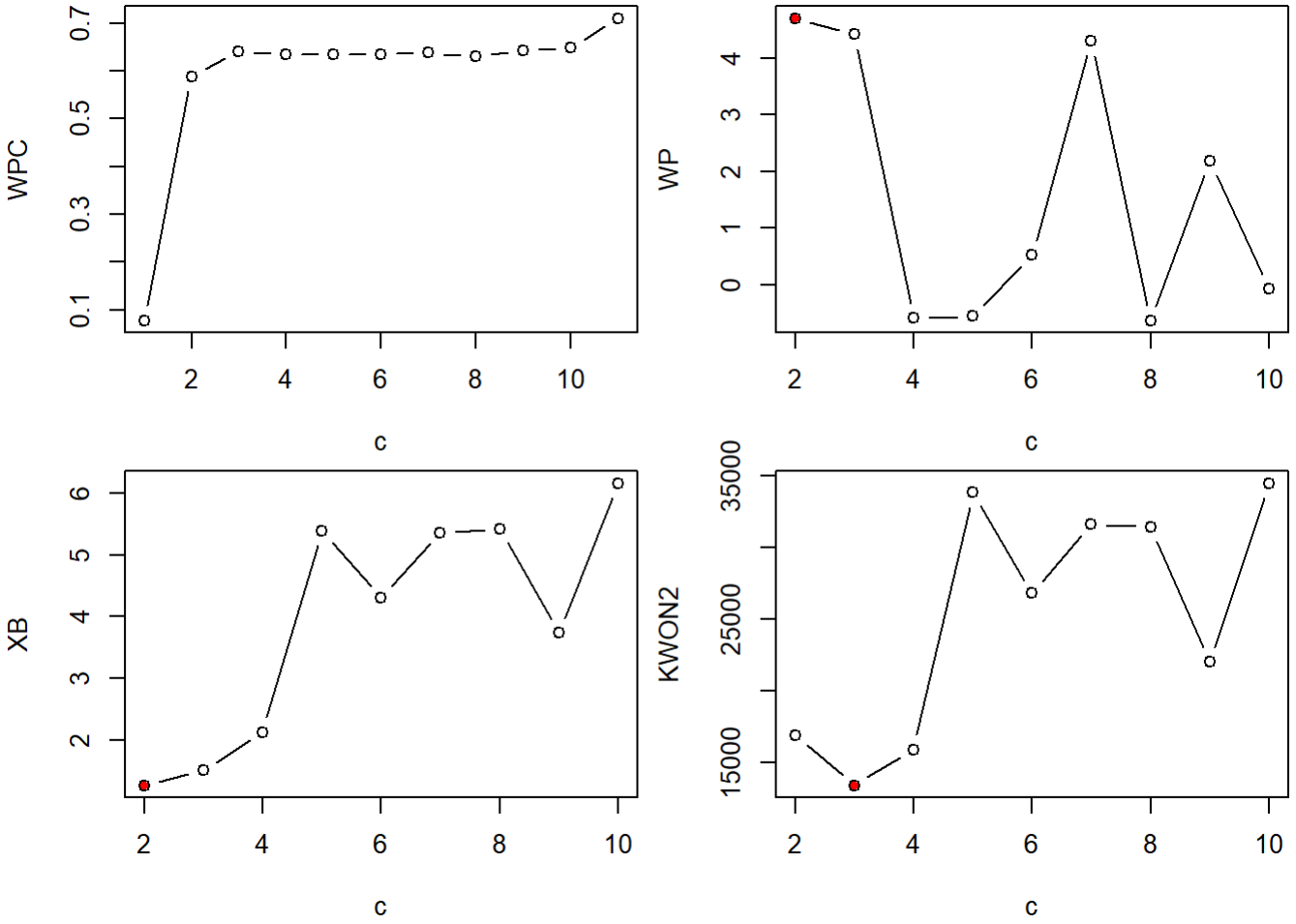
จาก Hvalid (kmeans) และ FzzyCVIs (EM) ได้ผลว่า kmeans เลือก 5 กลุ่ม จาก index = NCI ที่เป็นจุด Optimal EM เลือก 5 กลุ่ม จาก index = KWON2

```
set.seed(123)
plot_idx(Hvalid(texas_pca$x[,1:pc_select],
                kmin = 2, kmax = 10, nstart = 20,
                indexlist = c("NC", "NCI", "DB", "DBs"),
                method = "kmeans"))
```

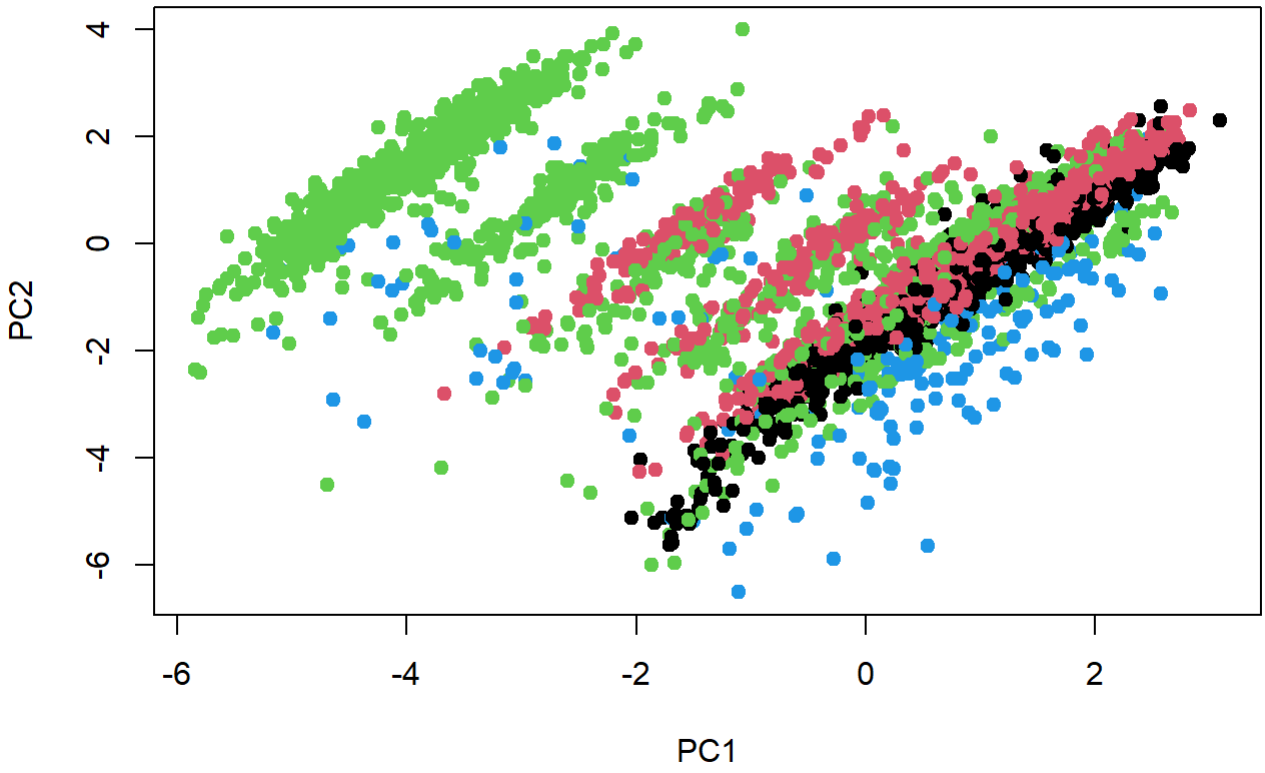
```
## Warning: did not converge in 10 iterations
## Warning: did not converge in 10 iterations
```



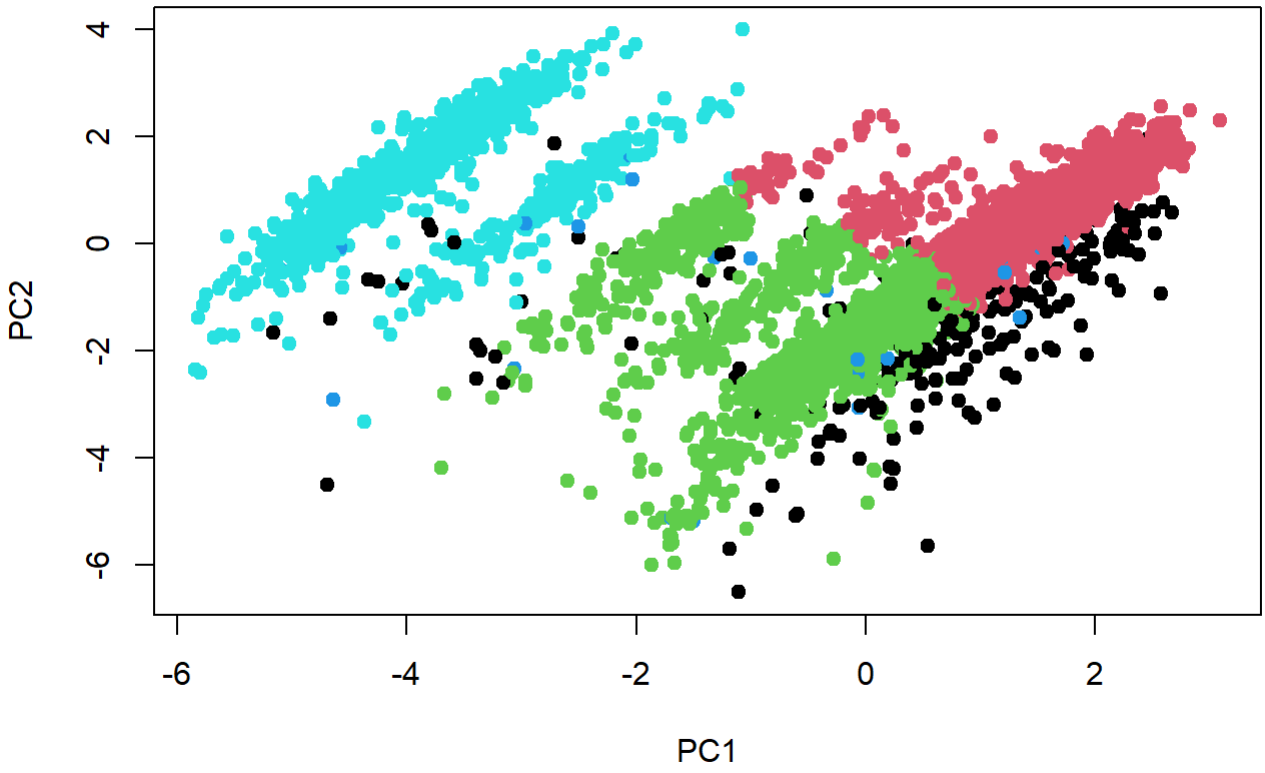

```
set.seed(123)
plot_idx(FzzyCVIs(texas_pca$x[,1:pc_select],
                cmin = 2, cmax = 10,
                indexlist = c("WPC", "WP", "XB","KWON2"),
                method = "EM"))
```




```
set.seed(123)
em <- Mclust(texas_pca$x[,1:pc_select], G = 4)
plot(texas_pca$x[,1:pc_select],
     col = em$classification,
     pch = 19,
     cex = 1)
```




```
set.seed(123)
km <- kmeans(texas_pca$x[,1:pc_select], 5)
plot(texas_pca$x[,1:pc_select],
     col = km$cluster,
     pch = 19,
     cex = 1)
```

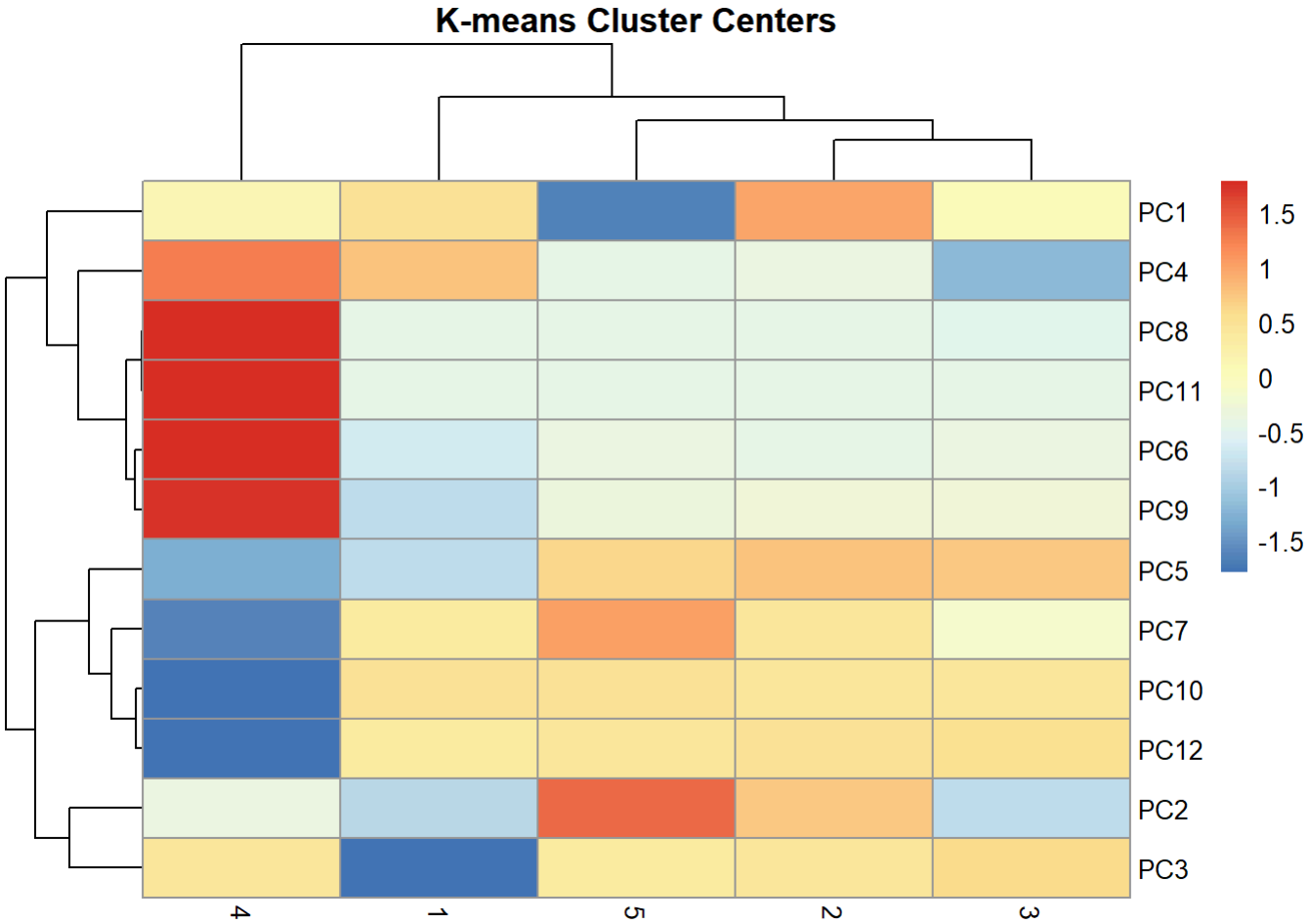



```
texas_clean$em_cluster <-  as.factor(em$classification)
texas_clean$km_cluster <-  as.factor(km$cluster)


km$centers
```

```
##          PC1        PC2        PC3         PC4         PC5         PC6
## 1  0.4512538 -1.5143599 -5.6308980  0.40352776 -1.14843584 -0.45256589
## 2  1.4427039  0.5034356  0.1379404  0.05863464  0.07984827 -0.03505191
## 3 -0.3548226 -1.4446973  0.5236279 -0.19148343  0.06214860 -0.00505693
## 4 -0.2543309 -0.8779700  0.1020635  0.56263457 -1.52454520  3.57480929
## 5 -3.7213720  1.2864463 -0.1187937  0.03529776 -0.02387298  0.02500526
##            PC7         PC8         PC9        PC10         PC11        PC12
## 1 -0.005735467 -0.06248362 -0.74188127  0.11113795 -0.003980651 -0.30258477
## 2  0.036275861 -0.02854197  0.01788372  0.04894213 -0.021863783  0.01537410
## 3 -0.170487281 -0.12400928  0.02173798 -0.03751078 -0.039510938  0.17392763
## 4 -0.636394498  5.62336487  2.82611313 -4.54022130  3.066018271 -5.17699413
## 5  0.210728259  0.02047149 -0.07750187  0.12420858 -0.023374246 -0.01202136
```

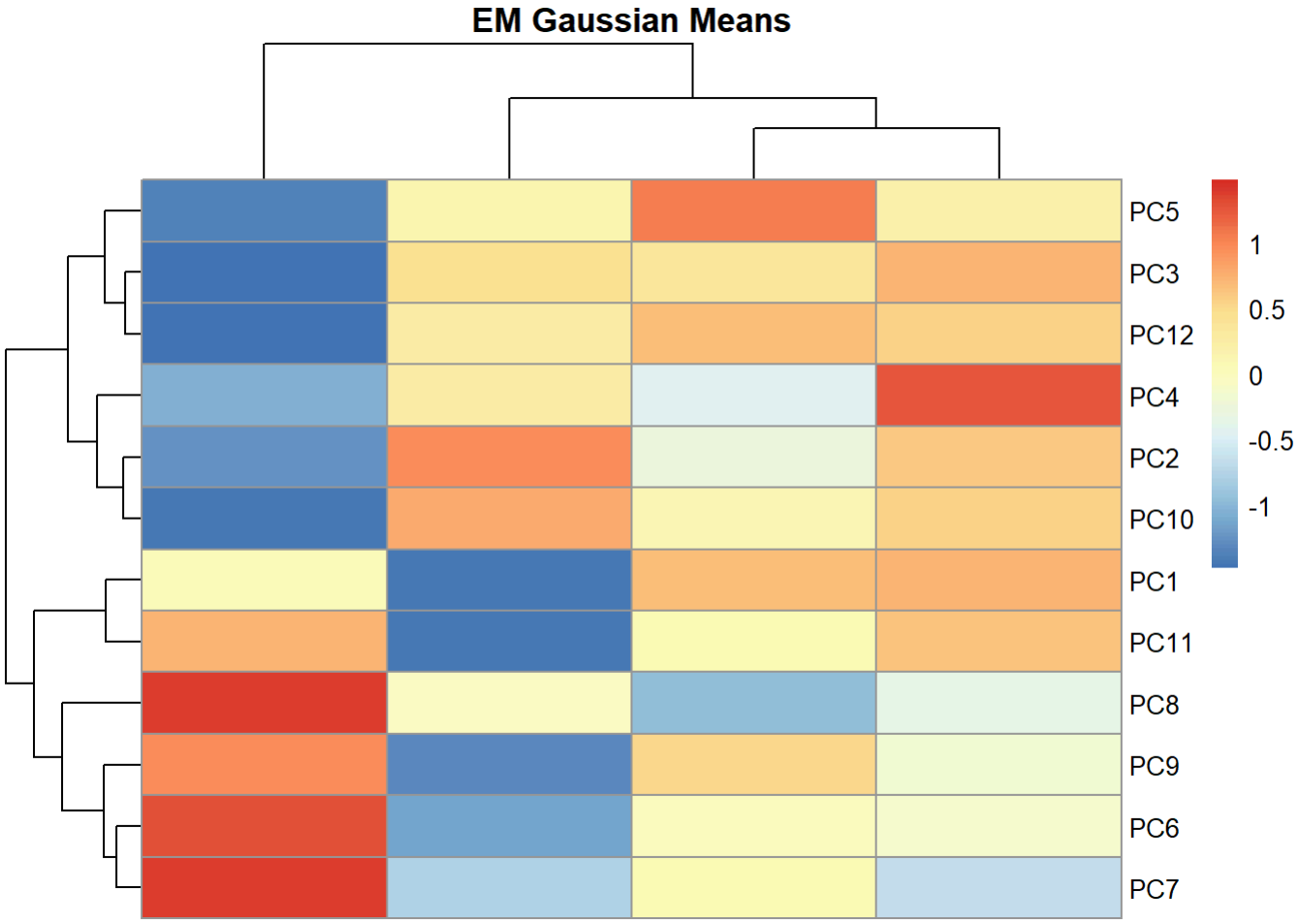
```
pheatmap(
  t(km$centers),
  scale = "row",
  clustering_method = "complete",
  main = "K-means Cluster Centers"
)
```




```
t(em$parameters$mean)
```

```
##               PC1        PC2         PC3         PC4         PC5        PC6
## [1,]  0.847189777 -0.6911202 -0.21485325 -0.33665234  0.66802770  0.1583184
## [2,]  0.961806273  0.1279536  0.41094942  0.26255375 -0.08485577  0.1008236
## [3,] -1.991210774  0.4756489 -0.04794535 -0.09647309 -0.12424166 -0.3549665
## [4,] -0.003362865 -1.5932240 -3.21121488 -0.54656285 -1.45495759  0.7632908
##               PC7         PC8         PC9        PC10        PC11       PC12
## [1,]  0.25737070 -0.14207611  0.11391384 -0.12182732  0.04815306  0.3625811
## [2,] -0.09379139 -0.01024505  0.02138425  0.03191484  0.19709907  0.1853062
## [3,] -0.14462458  0.06086040 -0.12966398  0.12275971 -0.35456102 -0.1900894
## [4,]  0.86877806  0.36788366  0.17156205 -0.68326574  0.22354477 -2.4780546
```

```
pheatmap(
  em$parameters$mean,
  scale = "row",
  clustering_method = "complete",
  main = "EM Gaussian Means"
)
```



จากการสรุปผล cluster เทียบกับ PCA ตีความกลุ่ม cluster ได้ดังนี้

ผล km_cluster

km_Cluster 1

PC1 = -0.387 → ต่ำ → อุณหภูมิ/Wind_Chill ต่ำ, twilight สูง

PC2 = -1.496 → ต่ำ → Humidity/Pressure สูง, Temperature/Chill ต่ำ

PC3 = -0.313 → ต่ำ → Bump/Traffic Calming ต่ำ → ถนนเรียบ/ไม่มีอุปสรรค

PC4 = 0.493 → สูง → จุดตัด/Give_Way มาก

PC5 = 0.291 → สูง → Visibility/amenity สูง

Cluster 1 = สภาพอากาศเย็น, ถนนเรียบ, มีจุดตัดเยอะ, amenity สูง

km_Cluster 2

PC1 = 0.849 → สูง → Temperature/Chill สูง, twilight ต่ำ

PC2 = 0.223 → สูงเล็กน้อย → Temperature/Chill สูง, Humidity/Pressure ต่ำ

PC4 = 0.778 → สูง → จุดตัด/Give_Way มาก

PC8 = -1.650 → ต่ำ → Junction/Stop/amenity ต่ำ

Cluster 2 = อากาศอุ่น, ถนนมีจุดตัดเยอะ, junction/stop/amenity น้อย

km_Cluster 3

PC1 = 1.400 → สูงมาก → อากาศอุ่น, twilight ต่ำ

PC2 = 0.442 → สูง → Temperature/Chill สูง, Humidity/Pressure ต่ำ

PC4 = 0.082 → ปานกลาง → จุดตัด/traffic average

PC8 = 0.150 → ปานกลาง → junction/stop/amenity ปานกลาง Cluster 3 = อากาศร้อน, ถนนธรรมดา, junction/stop ปานกลาง

Km_Cluster 4

PC1 = 0.416 → สูง → Temperature/Chill สูง, twilight ต่ำ

PC2 = -1.718 → ต่ำ → Humidity/Pressure สูง, Temperature ต่ำ

PC4 = -4.573 → ต่ำมาก → จุดตัด/Give_Way ต่ำ → ถนนเรียบ/ไม่มีจุดตัด

PC6 = 0.890 → สูง → Railway/Station/Stop สูง

Cluster 4 = อากาศเย็น, ถนนไม่มีจุดตัด, มี railway/station/stop เยอะ

Km_Cluster 5

PC1 = -3.742 → ต่ำมาก → Temperature/Chill ต่ำ, twilight สูง

PC2 = 1.233 → สูง → Temperature/Chill สูง, Humidity/Pressure ต่ำ (ขัดกัน PC1 vs PC2)

PC4 = -0.114 → ปานกลาง → จุดตัดน้อย

PC6 = 0.099 → ปานกลาง → Railway/Stop/Station ปานกลาง

Cluster 5 = อากาศเย็น/แปรปรวน, ถนนมีจุดตัดน้อย, railway/stop ปานกลาง

ผล em_cluster

EM Cluster 1 PC1 = 0.993 → สูง → Temperature/F, Wind_Chill สูง (อากาศร้อน/อบอุ่น), twilight ต่ำ PC2 = -0.135 → ต่ำเล็กน้อย → Humidity/Pressure สูง, Temperature ต่ำเล็กน้อย PC3 = -0.026 → ใกล้ 0 → Bump/Traffic Calming ปานกลาง PC4 = 0.200 → สูงเล็กน้อย → จุดตัด/Give_Way มาก PC5 = -0.072 → ต่ำ → Visibility/amenity ต่ำ EM Cluster 1 = อากาศอบอุ่น, ถนนธรรมดา, จุดตัดบางส่วน, visibility/amenity ต่ำ

EM Cluster 2 PC1 = -0.500 → ต่ำ → Temperature/Chill ต่ำ, twilight สูง → อากาศเย็น PC2 = -0.290 → ต่ำ → Humidity/Pressure สูง, Temperature ต่ำ → อากาศเย็น/ชื้น PC3 = -0.139 → ต่ำ → Bump/Traffic Calming ต่ำ → ถนนเรียบ/ไม่มีอุปสรรค PC5 = 0.417 → สูง → Visibility/amenity สูง PC7 = -1.033 → ต่ำ → Wind_Speed, Start_Lat/Lng ส่งผล EM Cluster 2 = อากาศเย็น/ชื้น, ถนนเรียบ, visibility/amenity สูง, พฤติกรรมลม/ตำแหน่งมีผล

EM Cluster 3 PC1 = -3.228 → ต่ำมาก → Temperature/Chill ต่ำ, twilight สูง → อากาศเย็นจัด PC2 = 1.007 → สูง → Temperature/Chill สูง → แสดงความแปรปรวนใน cluster PC3 = 0.134 → สูงเล็กน้อย → Bump/Traffic Calming สูง → มีอุปสรรคเล็กน้อย PC6 = 0.059 → ใกล้ 0 → Railway/Stop ปานกลาง EM Cluster 3 = อากาศเย็น/แปรปรวน, ถนนมีอุปสรรคปานกลาง, railway/stop ปานกลาง

EM Cluster 4 PC1 = 0.200 → ใกล้ 0 → Temperature/Chill ปานกลาง PC2 = -1.488 → ต่ำมาก → Humidity/Pressure สูง, Temperature ต่ำ → อากาศเย็น/ชื้น PC4 = -3.768 → ต่ำมาก → จุดตัด/Give_Way ต่ำ → ถนนเรียบ, จุดตัดน้อย PC6 = 1.173 → สูง → Railway/Stop/Station สูง → มีระบบขนส่งชัดเจน PC12 = -2.320 → ต่ำมาก → Civil/Nautical Twilight สูง → เวลากลางคืน/เช้ามีผล EM Cluster 4 = อากาศเย็น, ถนนเรียบ, จุดตัดน้อย, มี railway/station/stop เยอะ, twilight/เวลาของวัน

4. Supervise

Random Forest 5-folds XGboost 5-folds BART 5-folds

Split train/test

```
set.seed(123)
train_index <- sample(1:nrow(texas_clean), size = 0.8 * nrow(texas_clean))
train_texas <- texas_clean[train_index,]
test_texas <- texas_clean[-train_index,]
```

Random Forest Classification

```
set.seed(123)
zab <- train_texas
k <- 5
folds <- sample(1:k, nrow(zab), replace = TRUE)

# เก็บผลลัพธ์ของแต่ละ fold
precision_vec <- numeric(k)
recall_vec    <- numeric(k)
F1_vec        <- numeric(k)
accuracy_vec  <- numeric(k)

for(i in 1:k){
  train_fold <- zab[folds != i, ]
  test_fold  <- zab[folds == i, ]

  rf_mod <- randomForest(x = train_fold[, -1], y = train_fold$Severity, ntree = 300)

  pred <- predict(rf_mod, test_fold[, -1])

  cm <- table(Predicted = pred, Actual = test_fold$Severity)

  # ตรวจว่า TP/TN/FN/FP มีครบ
  TP <- ifelse("1" %in% rownames(cm) & "1" %in% colnames(cm), cm["1","1"], 0)
  FP <- ifelse("1" %in% rownames(cm) & "0" %in% colnames(cm), cm["1","0"], 0)
  FN <- ifelse("0" %in% rownames(cm) & "1" %in% colnames(cm), cm["0","1"], 0)
  TN <- ifelse("0" %in% rownames(cm) & "0" %in% colnames(cm), cm["0","0"], 0)

  precision_vec[i] <- ifelse(TP+FP==0, NA, TP/(TP+FP))
  recall_vec[i]    <- ifelse(TP+FN==0, NA, TP/(TP+FN))
  F1_vec[i]        <- ifelse(is.na(precision_vec[i]) | is.na(recall_vec[i]), NA, 2*precision_
vec[i]*recall_vec[i]/(precision_vec[i]+recall_vec[i]))
  accuracy_vec[i]  <- (TP+TN)/sum(cm)
}
```

```
## Warning in randomForest.default(x = train_fold[, -1], y = train_fold$Severity,
## : The response has five or fewer unique values.  Are you sure you want to do
## regression?
## Warning in randomForest.default(x = train_fold[, -1], y = train_fold$Severity,
## : The response has five or fewer unique values.  Are you sure you want to do
## regression?
## Warning in randomForest.default(x = train_fold[, -1], y = train_fold$Severity,
## : The response has five or fewer unique values.  Are you sure you want to do
## regression?
## Warning in randomForest.default(x = train_fold[, -1], y = train_fold$Severity,
## : The response has five or fewer unique values.  Are you sure you want to do
## regression?
## Warning in randomForest.default(x = train_fold[, -1], y = train_fold$Severity,
## : The response has five or fewer unique values.  Are you sure you want to do
## regression?
```

```
# ค่าเฉลี่ยจากทุก fold
rf_metrics <- list(
  Precision = mean(precision_vec, na.rm=TRUE),
  Recall    = mean(recall_vec, na.rm=TRUE),
  F1        = mean(F1_vec, na.rm=TRUE),
  Accuracy  = mean(accuracy_vec, na.rm=TRUE)
)
```

XGBoost Classification

```
set.seed(123)
k <- 5
folds <- sample(1:k, nrow(train_texas), replace = TRUE)

precision_vec <- numeric(k)
recall_vec    <- numeric(k)
F1_vec        <- numeric(k)
accuracy_vec  <- numeric(k)

for(i in 1:k){
  train_fold <- train_texas[folds != i, ]
  test_fold  <- train_texas[folds == i, ]

  # แปลง train เป็น matrix พร้อม dummy สำหรับ em_cluster & km_cluster
  train_x <- model.matrix(~ . - Severity - Start_Lat - Start_Lng, data = train_fold)[,-1] # r
emove intercept
  train_y <- as.numeric(train_fold$Severity)

  test_x <- model.matrix(~ . - Severity - Start_Lat - Start_Lng, data = test_fold)[,-1]
  test_y <- as.numeric(test_fold$Severity)

  dtrain <- xgb.DMatrix(data = train_x, label = train_y)
  dtest  <- xgb.DMatrix(data = test_x,  label = test_y)

  params <- list(objective = "binary:logistic",
                 eval_metric = "logloss",
                 eta = 0.2,
                 max_depth = 8)

  xgb_mod <- xgb.train(params=params, data=dtrain, nrounds=200, verbose=0)

  pred_prob <- predict(xgb_mod, dtest)
  pred <- ifelse(pred_prob > 0.25, 1, 0)

  cm <- table(Predicted = pred, Actual = test_y)

  TP <- ifelse("1" %in% rownames(cm) & "1" %in% colnames(cm), cm["1","1"], 0)
  FP <- ifelse("1" %in% rownames(cm) & "0" %in% colnames(cm), cm["1","0"], 0)
  FN <- ifelse("0" %in% rownames(cm) & "1" %in% colnames(cm), cm["0","1"], 0)
  TN <- ifelse("0" %in% rownames(cm) & "0" %in% colnames(cm), cm["0","0"], 0)

  precision_vec[i] <- ifelse(TP+FP==0, NA, TP/(TP+FP))
  recall_vec[i]    <- ifelse(TP+FN==0, NA, TP/(TP+FN))
  F1_vec[i]        <- ifelse(is.na(precision_vec[i]) | is.na(recall_vec[i]), NA, 2*precision_
vec[i]*recall_vec[i]/(precision_vec[i]+recall_vec[i]))
  accuracy_vec[i]  <- (TP+TN)/sum(cm)
}

# ค่า metric เฉลี่ยจากทุก fold
xgb_metrics <- list(
  Precision = mean(precision_vec, na.rm=TRUE),
  Recall    = mean(recall_vec, na.rm=TRUE),
  F1        = mean(F1_vec, na.rm=TRUE),
```

```
  Accuracy  = mean(accuracy_vec, na.rm=TRUE)
)
```

BART Classification

```
set.seed(123)
k <- 5
folds <- sample(1:k, nrow(train_texas), replace = TRUE)

precision_vec <- numeric(k)
recall_vec    <- numeric(k)
F1_vec        <- numeric(k)
accuracy_vec  <- numeric(k)

for(i in 1:k){
  train_fold <- train_texas[folds != i, ]
  test_fold  <- train_texas[folds == i, ]

  # แปลง factor -> numeric
  train_x <- train_fold |>
    select(-Severity,-Start_Lat,-Start_Lng) |>
    mutate(
      em_cluster = as.numeric(as.factor(em_cluster)),
      km_cluster = as.numeric(as.factor(km_cluster))
    ) |>
    as.matrix()
  train_y <- as.numeric(train_fold$Severity)

  test_x <- test_fold |>
    select(-Severity,-Start_Lat,-Start_Lng) |>
    mutate(
      em_cluster = as.numeric(as.factor(em_cluster)),
      km_cluster = as.numeric(as.factor(km_cluster))
    ) |>
    as.matrix()
  test_y <- as.numeric(test_fold$Severity)

  bart_mod <- pbart(x.train=train_x, y.train=train_y, x.test=test_x,
                    nskip=200, ndpost=500, ntree=400)

  pred_prob <- bart_mod$prob.test.mean
  pred <- ifelse(pred_prob > 0.25, 1, 0)

  cm <- table(Predicted = pred, Actual = test_y)

  TP <- ifelse("1" %in% rownames(cm) & "1" %in% colnames(cm), cm["1","1"], 0)
  FP <- ifelse("1" %in% rownames(cm) & "0" %in% colnames(cm), cm["1","0"], 0)
  FN <- ifelse("0" %in% rownames(cm) & "1" %in% colnames(cm), cm["0","1"], 0)
  TN <- ifelse("0" %in% rownames(cm) & "0" %in% colnames(cm), cm["0","0"], 0)

  precision_vec[i] <- ifelse(TP+FP==0, NA, TP/(TP+FP))
  recall_vec[i]    <- ifelse(TP+FN==0, NA, TP/(TP+FN))
  F1_vec[i]        <- ifelse(is.na(precision_vec[i]) | is.na(recall_vec[i]), NA, 2*precision_
vec[i]*recall_vec[i]/(precision_vec[i]+recall_vec[i]))
  accuracy_vec[i]  <- (TP+TN)/sum(cm)
}
```

```
## *****Into main of pbart
## *****Data:
## data:n,p,np: 3421, 22, 865
## y1,yn: 0, 1
## x1,x[n*p]: 78.000000, 2.000000
## xp1,xp[np*p]: 71.000000, 5.000000
## *****Number of Trees: 400
## *****Number of Cut Points: 91 ... 4
## *****burn and ndpost: 200, 500
## *****Prior:mybeta,alpha,tau: 2.000000,0.950000,0.075000
## *****binaryOffset: -0.300862
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,22,0
## *****nkeeptrain,nkeeptest,nkeeptreedraws: 500,500,500
## *****printevery: 100
## *****skiptr,skipte,skiptreedraws: 1,1,1
##
## MCMC
## done 0 (out of 700)
## done 100 (out of 700)
## done 200 (out of 700)
## done 300 (out of 700)
## done 400 (out of 700)
## done 500 (out of 700)
## done 600 (out of 700)
## time: 28s
## check counts
## trcnt,tecnt: 500,500
## *****Into main of pbart
## *****Data:
## data:n,p,np: 3377, 22, 909
## y1,yn: 0, 0
## x1,x[n*p]: 78.000000, 5.000000
## xp1,xp[np*p]: 69.000000, 3.000000
## *****Number of Trees: 400
## *****Number of Cut Points: 91 ... 4
## *****burn and ndpost: 200, 500
## *****Prior:mybeta,alpha,tau: 2.000000,0.950000,0.075000
## *****binaryOffset: -0.304128
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,22,0
## *****nkeeptrain,nkeeptest,nkeeptreedraws: 500,500,500
## *****printevery: 100
## *****skiptr,skipte,skiptreedraws: 1,1,1
##
## MCMC
## done 0 (out of 700)
## done 100 (out of 700)
## done 200 (out of 700)
## done 300 (out of 700)
## done 400 (out of 700)
## done 500 (out of 700)
## done 600 (out of 700)
## time: 24s
## check counts
## trcnt,tecnt: 500,500
## *****Into main of pbart
```

```
## *****Data:
## data:n,p,np: 3444, 22, 842
## y1,yn: 0, 0
## x1,x[n*p]: 69.000000, 5.000000
## xp1,xp[np*p]: 78.000000, 2.000000
## *****Number of Trees: 400
## *****Number of Cut Points: 92 ... 4
## *****burn and ndpost: 200, 500
## *****Prior:mybeta,alpha,tau: 2.000000,0.950000,0.075000
## *****binaryOffset: -0.280954
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,22,0
## *****nkeeptrain,nkeeptest,nkeeptreedraws: 500,500,500
## *****printevery: 100
## *****skiptr,skipte,skiptreedraws: 1,1,1
##
## MCMC
## done 0 (out of 700)
## done 100 (out of 700)
## done 200 (out of 700)
## done 300 (out of 700)
## done 400 (out of 700)
## done 500 (out of 700)
## done 600 (out of 700)
## time: 25s
## check counts
## trcnt,tecnt: 500,500
## *****Into main of pbart
## *****Data:
## data:n,p,np: 3444, 22, 842
## y1,yn: 0, 0
## x1,x[n*p]: 78.000000, 5.000000
## xp1,xp[np*p]: 50.000000, 5.000000
## *****Number of Trees: 400
## *****Number of Cut Points: 92 ... 4
## *****burn and ndpost: 200, 500
## *****Prior:mybeta,alpha,tau: 2.000000,0.950000,0.075000
## *****binaryOffset: -0.306030
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,22,0
## *****nkeeptrain,nkeeptest,nkeeptreedraws: 500,500,500
## *****printevery: 100
## *****skiptr,skipte,skiptreedraws: 1,1,1
##
## MCMC
## done 0 (out of 700)
## done 100 (out of 700)
## done 200 (out of 700)
## done 300 (out of 700)
## done 400 (out of 700)
## done 500 (out of 700)
## done 600 (out of 700)
## time: 24s
## check counts
## trcnt,tecnt: 500,500
## *****Into main of pbart
## *****Data:
## data:n,p,np: 3458, 22, 828
```

```
## y1,yn: 0, 0
## x1,x[n*p]: 78.000000, 5.000000
## xp1,xp[np*p]: 55.000000, 2.000000
## *****Number of Trees: 400
## *****Number of Cut Points: 87 ... 4
## *****burn and ndpost: 200, 500
## *****Prior:mybeta,alpha,tau: 2.000000,0.950000,0.075000
## *****binaryOffset: -0.300957
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,22,0
## *****nkeeptrain,nkeeptest,nkeeptreedraws: 500,500,500
## *****printevery: 100
## *****skiptr,skipte,skiptreedraws: 1,1,1
##
## MCMC
## done 0 (out of 700)
## done 100 (out of 700)
## done 200 (out of 700)
## done 300 (out of 700)
## done 400 (out of 700)
## done 500 (out of 700)
## done 600 (out of 700)
## time: 24s
## check counts
## trcnt,tecnt: 500,500
```

```
# ค่า metric เฉลี่ยจากทุก fold
bart_metrics <- list(
  Precision = mean(precision_vec, na.rm=TRUE),
  Recall    = mean(recall_vec, na.rm=TRUE),
  F1        = mean(F1_vec, na.rm=TRUE),
  Accuracy  = mean(accuracy_vec, na.rm=TRUE)
)
```

ตาราง สรุปผล

```
summary_df <- bind_rows(
  RandomForest = as.data.frame(rf_metrics),
  XGBoost      = as.data.frame(xgb_metrics),
  BART         = as.data.frame(bart_metrics),
  .id = "Model"
)

# ดูผลลัพธ์
summary_df
```

```
##          Model Precision    Recall        F1  Accuracy
## 1 RandomForest       NaN       NaN       NaN 0.0000000
## 2      XGBoost 0.4374697 0.6432008 0.5205452 0.5471329
## 3         BART 0.4008042 0.9649043 0.5662646 0.4350407
```

RESULT

. . .

จากการที่ XGboost ให้ผล F1-score มากที่สุด และ ได้ accuracy ที่เกิน 50% จึงจะใช้ Xgboost ในการศึกษาตัวแปรต่อไป

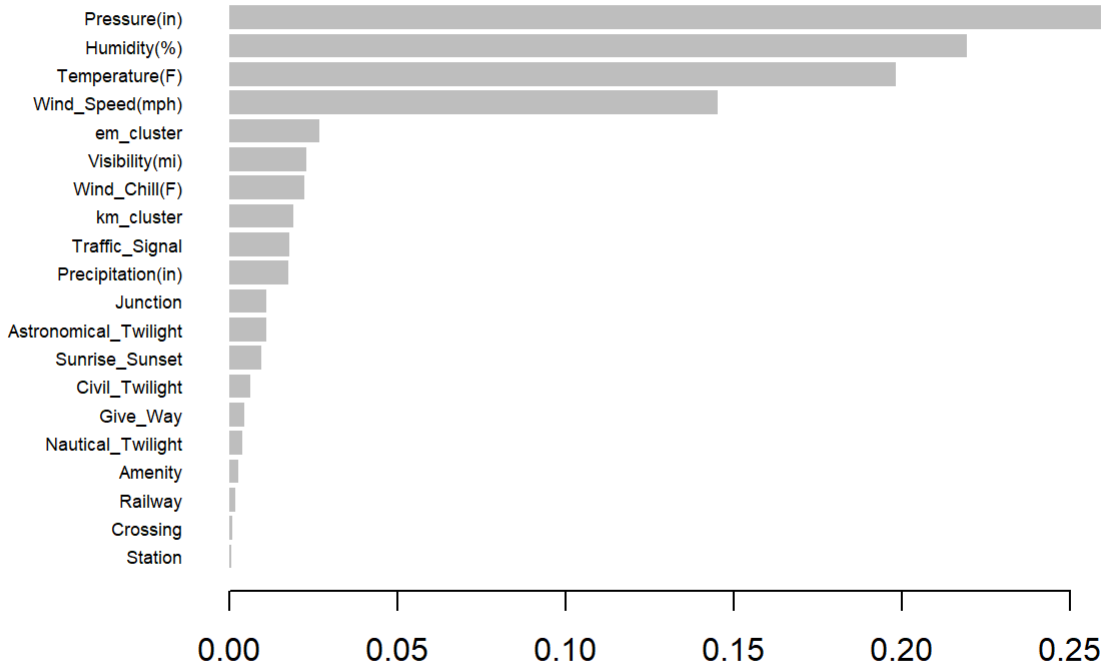
5. ดู feature importance ด้วย SHAP

```
# Train final model
dtrain_full <- xgb.DMatrix(data = train_x, label = train_y)
params <- list(objective = "binary:logistic",
               eval_metric = "logloss",
               eta = 0.2,
               max_depth = 8)

xgb_final <- xgb.train(params = params, data = dtrain_full, nrounds=200, verbose=0)

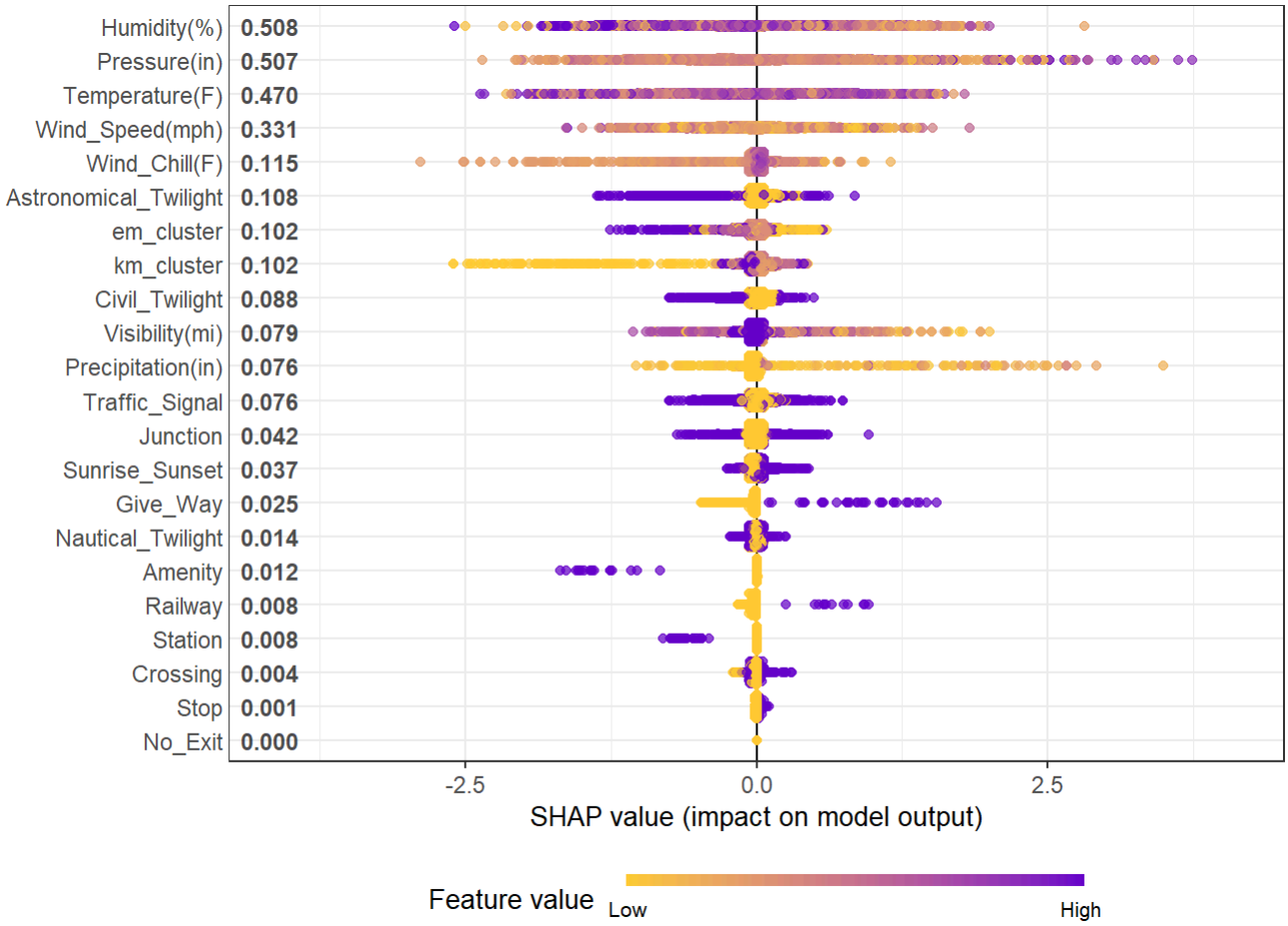
# SHAP values
shap_values <- xgb.importance(feature_names = colnames(train_x), model = xgb_final)

# ดู top 20 features
xgb.plot.importance(shap_values[1:20,])
```




```
# สร้าง SHAP matrix
shap_long <- shap.prep(xgb_model = xgb_final, X_train = train_x)

# summary plot
shap.plot.summary(shap_long)
```



ได้ผลว่า top 5 ตัวแปรสำคัญที่ model ใช้ในการทำนายคือ 1.) Humidity(%) ความชื้นสัมพัทธ์ (%) โดย ยิ่งความชื้นสัมพัทธ์ “น้อย” โอกาสที่จะเกิดอุบัติเหตุรุนแรงจะมีมากขึ้น 2.) Pressure(in) ความดันอากาศ (นิ้วปรอท) แม้จะไม่สามารถตีความได้อย่างชัดเจน แต่จะพบว่า เมื่อเจอค่าความดันที่“สูง” โอกาสที่จะเกิดอุบัติเหตุรุนแรงจะมีมากขึ้น 3.) Temperature(F) อุณหภูมิ (ฟาเรนไฮต์) ยิ่ง อุณหภูมิ “สูง” โอกาสที่จะเกิดอุบัติเหตุรุนแรงจะมีมากขึ้น 4.) Wind_Speed(mph) ความเร็วลม (ไมล์ต่อชั่วโมง) ยิ่ง ความเร็วลม “สูง” โอกาสที่จะเกิดอุบัติเหตุรุนแรงจะยิ่งมากขึ้น 5.) Traffic_Signal มีสัญญาณไฟจราจรอยู่ใกล้หรือไม่ ได้ว่า มีความคลุมเครือ ทั้ง เมื่อมีไม่มีสัญญาณไฟ(สีเหลือง) และ เมื่อมีสัญญาณไฟ (สีน้ำเงิน) ทำให้ xgboost เลือกที่จะทำนาย class ในสัดส่วนที่พอกัน โดยจากการตีความเพิ่มเติม จะพบว่า เมื่อมี traffic signal อุบัติเหตุจะมีทั้งรุนแรงและไม่รุนแรง หากมองในมุมของบริบทความเป็นจริง จุดที่มี traffic signal มักเป็นจุดสำคัญที่อาจเคยเกิดอุบัติเหตุบ่อย หรือเป็นจุดสำคัญที่ รัฐเล็งเห็นถึงความเสี่ยง จึงต้องมีการสร้างสัญญาณไป และเมื่อมีสัญญาณไฟความระมัดระวังของการขับขี่จะสูงมากยิ่งขึ้น แต่ในขณะเดียวกัน ก็จะมีผู้ที่ไม่ได้เคารพกฎการจราจรหรืออาจเป็นจุดที่เสี่ยงเกิดอุบัติเหตุอยู่แล้ว (ตามที่รัฐเห็นปัญหา) ณ จุดนี้ model จึง 50/50 ในการทำนาย

6. Waittime จุดสำคัญ

จากการทำนายผลและวิเคราะห์ตัวแปรแล้ว จะเลือก จุด 3 จุด ที่เทียบเคียงโอกาสเกิดอุบัติเหตุรุนแรงได้ 3 จุด คือ 1.) Junction อยู่ใกล้สี่แยกหรือไม่ 2.) Traffic_Signal มีสัญญาณไฟจราจรอยู่ใกล้หรือไม่ 3.) Crossing มีทางข้ามถนนอยู่ใกล้หรือไม่ โดยจะนำจุดเกิดเหตุเหล่านี้มาใช้ในการหา waittime ในการเกิดอุบัติเหตุในครั้งต่อไป

```r
compute_wait_time <- function(df) {
  df %>%
    mutate(wait_hours = as.numeric(difftime(End_Time, Start_Time, units = "hours")))
}

# แยกตามจุดสำคัญ
junc_df <- dat %>% filter(Junction == 1)
traffic_df <- dat %>% filter(Traffic_Signal == 1)
crossing_df <- dat %>% filter(Crossing == 1)

# คำนวณ wait time
junc_df <- compute_wait_time(junc_df)
traffic_df <- compute_wait_time(traffic_df)
crossing_df <- compute_wait_time(crossing_df)

# ฟังก์ชันสรุป wait time และเพิ่ม column ประเภทจุด
summarize_wait <- function(df, type_name) {
  df %>%
    summarize(
      count = n(),
      mean_wait = mean(wait_hours, na.rm = TRUE),
      median_wait = median(wait_hours, na.rm = TRUE),
      sd_wait = sd(wait_hours, na.rm = TRUE),
      min_wait = min(wait_hours, na.rm = TRUE),
      max_wait = max(wait_hours, na.rm = TRUE)
    ) %>%
    mutate(point_type = type_name) %>%
    select(point_type, everything())
}

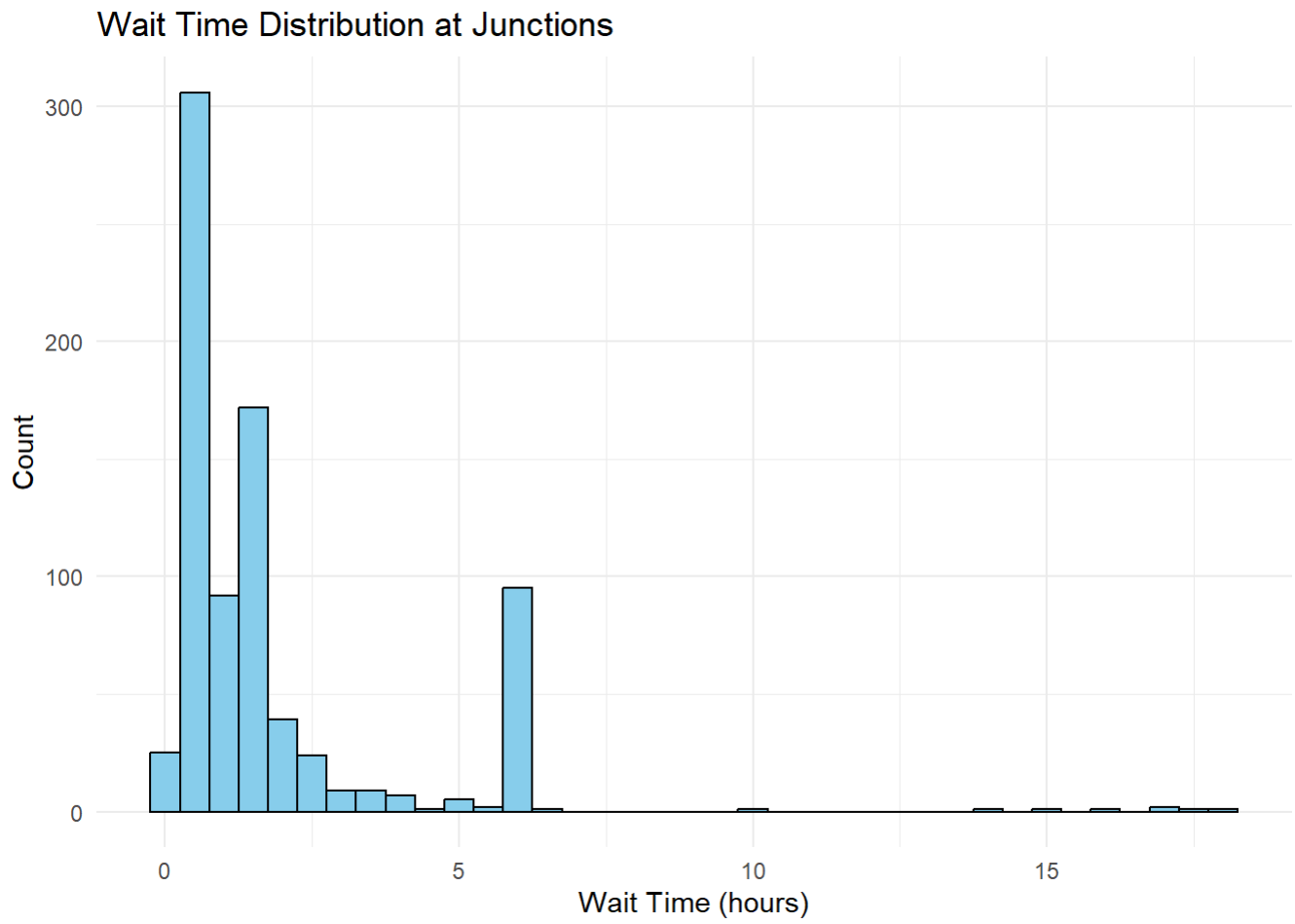
# สร้าง data frame ของแต่ละประเภท
junc_summary <- summarize_wait(junc_df, "Junction")
traffic_summary <- summarize_wait(traffic_df, "Traffic_Signal")
crossing_summary <- summarize_wait(crossing_df, "Crossing")

# รวมเป็น data frame เดียว
wait_summary_df <- bind_rows(junc_summary, traffic_summary, crossing_summary)

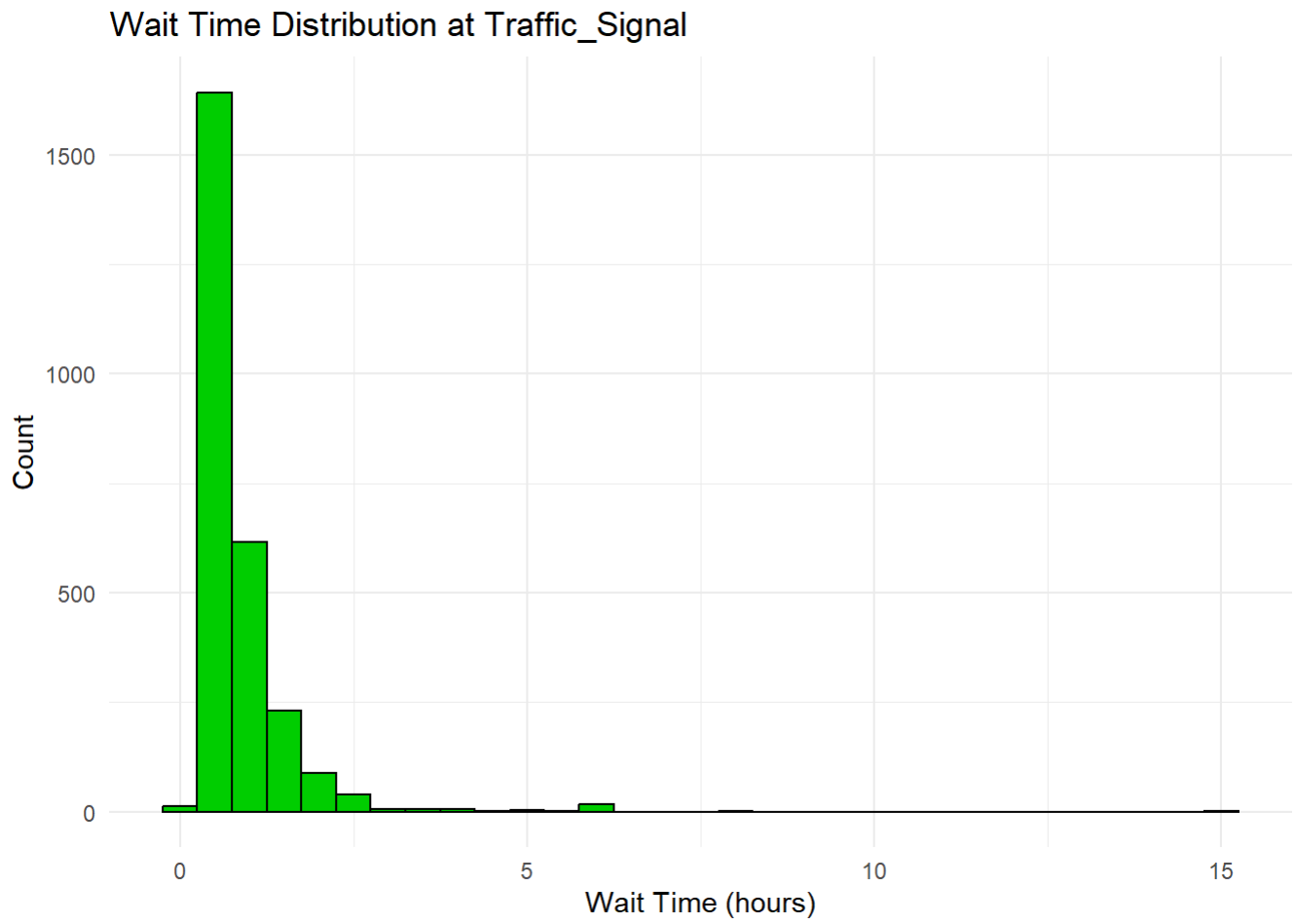
# ดูผลลัพธ์
wait_summary_df
```

```
## # A tibble: 3 × 7
##   point_type     count mean_wait median_wait sd_wait min_wait max_wait
##   <chr>          <int>     <dbl>       <dbl>   <dbl>    <dbl>    <dbl>
## 1 Junction         795     1.85        1.02    2.27     0.1       18.2
## 2 Traffic_Signal  2672     0.883       0.744   0.725    0.183     15.2
## 3 Crossing         308     0.915       0.746   0.750    0.25       8
```

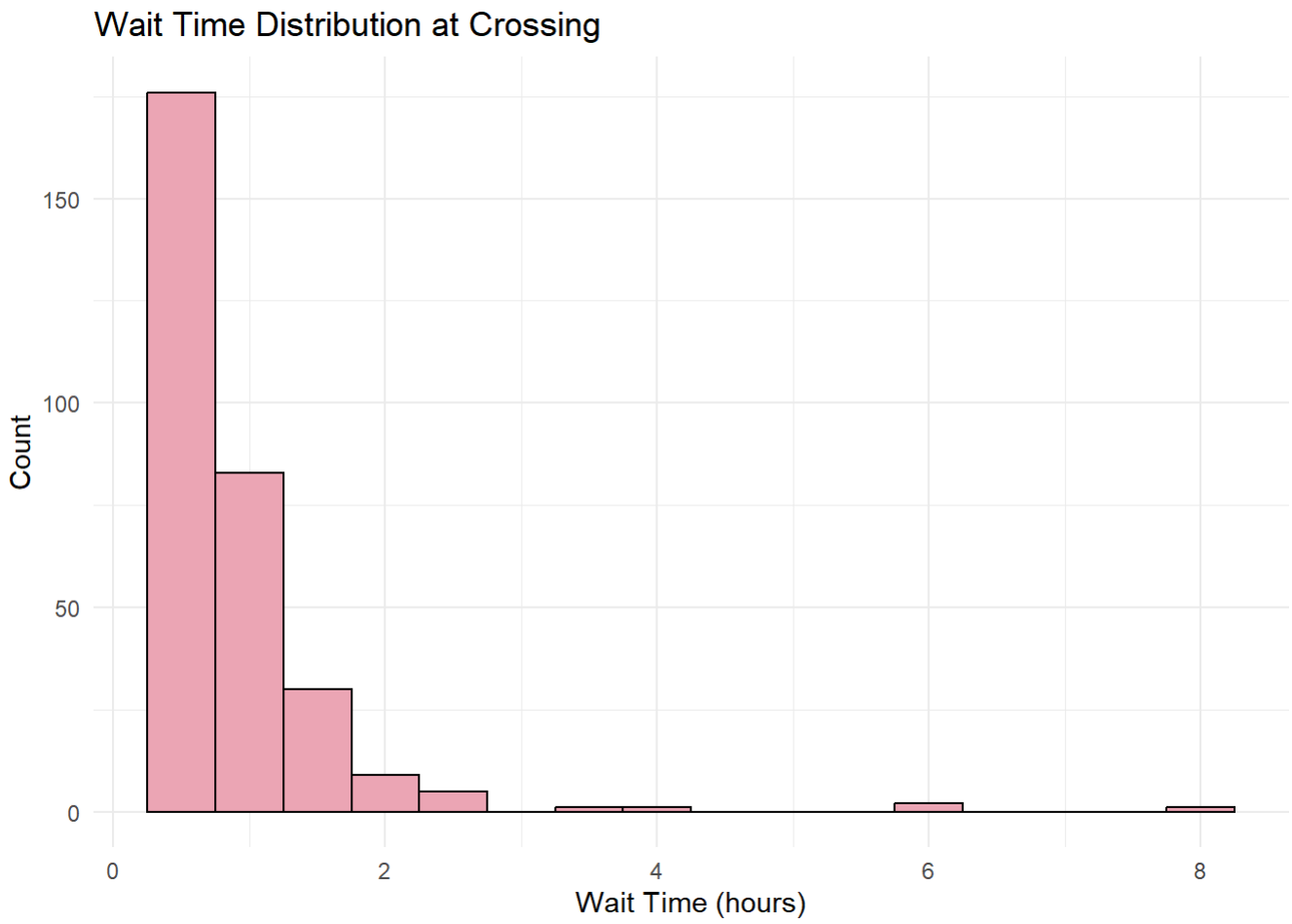
```r
ggplot(junc_df, aes(x = wait_hours)) +
  geom_histogram(binwidth = 0.5, fill="skyblue", color="black") +
  labs(title="Wait Time Distribution at Junctions", x="Wait Time (hours)", y="Count") +
  theme_minimal()
```




```
ggplot(traffic_df, aes(x = wait_hours)) +
  geom_histogram(binwidth = 0.5, fill="green3", color="black") +
  labs(title="Wait Time Distribution at Traffic_Signal", x="Wait Time (hours)", y="Count") +
  theme_minimal()
```




```
ggplot(crossing_df, aes(x = wait_hours)) +
  geom_histogram(binwidth = 0.5, fill="pink2", color="black") +
  labs(title="Wait Time Distribution at Crossing", x="Wait Time (hours)", y="Count") +
  theme_minimal()
```


Wait Time Distribution at Crossing
Count
150
100
50
0
0
2
4
6
8
Wait Time (hours)